

NAME: SHREYA GHOLASE

PRN NO: 23070521145

Lab 8

PL/SQL Procedure for Fund Transfer

Step 1: Create Database Tables

1.1 Create `accounts` Table

```
CREATE TABLE accounts (  
    account_no NUMBER PRIMARY KEY,  
    holder_name VARCHAR2(100),  
    balance NUMBER(10,2) CHECK (balance >= 0)  
);
```

```
SQL> CREATE TABLE accounts (  
2  account_no NUMBER PRIMARY KEY,  
3  holder_name VARCHAR2(100),  
4  balance NUMBER(10,2) CHECK (balance >= 0)  
5  );
```

Table created.

1.2 Create `transactions` Table

```
CREATE TABLE transactions (  
    transaction_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY
```

```
KEY,  
    from_account NUMBER,  
  
    amount NUMBER(10,2),  
    transaction_date TIMESTAMP DEFAULT SYSTIMESTAMP  
);
```

```
SQL> CREATE TABLE transactions (  
2   transaction_id NUMBER primary key,  
3   from_account NUMBER,  
4   amount NUMBER(10,2),  
5   transaction_date TIMESTAMP DEFAULT SYSTIMESTAMP  
6   );
```

Table created.

Step 2: Insert Sample Data

```
INSERT INTO accounts VALUES (101, 'Alice', 5000.00);  
INSERT INTO accounts VALUES (102, 'Bob', 3000.00);  
COMMIT;
```

```

SQL> INSERT INTO accounts VALUES (101, 'Alice', 5000.00);
1 row created.

SQL> INSERT INTO accounts VALUES (102, 'Bob', 3000.00); COMMIT;
INSERT INTO accounts VALUES (102, 'Bob', 3000.00); COMMIT
*
ERROR at line 1:
ORA-00911: invalid character

SQL> INSERT INTO accounts VALUES (102, 'Bob', 3000.00);
1 row created.

SQL> commit ;
Commit complete.

```

Step 3: Write PL/SQL Procedure

```

CREATE OR REPLACE PROCEDURE transfer_funds(
    p_to_acc NUMBER,
    p_amount NUMBER
) AS
    v_balance NUMBER;
BEGIN
    -- Check if sender has sufficient balance
    SELECT balance INTO v_balance FROM accounts WHERE account_no =
p_from_acc;

    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient balance.');
```

```

        -- Deduct amount from sender
        UPDATE accounts SET balance = balance - p_amount WHERE account_no
= p_from_acc;

        -- Add amount to receiver
        UPDATE accounts SET balance = balance + p_amount WHERE account_no
= p_to_acc;

        -- Log transaction
        INSERT INTO transactions (from_account, to_account, amount)
        VALUES (p_from_acc, p_to_acc, p_amount);

        -- Commit transaction
        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Transfer successful.');
```

EXCEPTION

```

        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20002, 'Invalid account number.');
```

WHEN OTHERS THEN

```

            ROLLBACK;
            RAISE_APPLICATION_ERROR(-20003, 'Transaction failed: ' ||
SQLERRM);
END;
/
```

```

SQL> CREATE OR REPLACE PROCEDURE transfer_funds(
  2  p_to_acc NUMBER, --account number where money is sent.
  3  p_amount NUMBER -- amount of money to be transfer
  4  )
  5  as
  6  v_balance number; variable to store balance.
  7  BEGIN
  8  -- Check if sender has sufficient balance
  9  SELECT balance INTO v_balance FROM accounts WHERE account_no = p_from_a
cc;
10  IF v_balance < p_amount THEN
11  RAISE_APPLICATION_ERROR(-20001, 'Insufficient balance.');
```

```

12  END IF;
13  -- Deduct amount from sender
14  UPDATE accounts SET balance = balance - p_amount WHERE account_no = p_f
rom_acc;
15  -- Add amount to receiver
16  UPDATE accounts SET balance = balance + p_amount WHERE account_no = p_t
o_acc;
17  -- Log transaction
18  INSERT INTO transactions (from_account, to_account, amount) VALUES (p_f
rom_acc, p_to_acc, p_amount);
19  -- Commit transaction
20  COMMIT;
21  DBMS_OUTPUT.PUT_LINE('Transfer successful.');
```

```

22  EXCEPTION
23  WHEN NO_DATA_FOUND THEN
24  RAISE_APPLICATION_ERROR(-20002, 'Invalid account number.');
```

```

25  WHEN OTHERS THEN
26  ROLLBACK;
27  RAISE_APPLICATION_ERROR(-20003, 'Transaction failed: ' || SQLERRM);
28  END;
29  /

```

Warning: Procedure created with compilation errors.

- First, it checks if the sender has enough money before making the transfer.
- If the balance is sufficient, it deducts the amount from the sender's account and adds it to the receiver's account.
- It also keeps a record of the transaction for future reference.
- The procedure includes error handling to deal with cases like insufficient funds, invalid account numbers, or unexpected issues

Step 4: Execute Procedure

```

BEGIN
    transfer_funds(101, 102, 1000);

```

```
END;  
/
```

```
SQL> BEGIN  
  2  transfer_funds(101, 102, 1000);  
  3  END;  
  4  /  
  
PL/SQL procedure successfully completed.
```

- -This block runs the procedure we created earlier, providing the necessary input values to make it work

Step 5: Verify Results

Check Account Balances

```
SELECT * FROM accounts;
```

```
SQL> SELECT * FROM accounts;
```

```
ACCOUNT_NO
```

```
-----
```

```
HOLDER_NAME
```

```
-----
```

```
----
```

```
BALANCE
```

```
-----
```

```
101
```

```
Alice
```

```
5000
```

```
102
```

```
Bob
```

```
3000
```

```
ACCOUNT_NO
```

```
-----
```

```
HOLDER_NAME
```

```
-----
```

```
----
```

```
BALANCE
```

```
-----
```

Check Transactions Log

```
SELECT * FROM transactions;
```

```
SQL> SELECT * FROM transactions;
```

```
no rows selected
```

Task: Fund Transfer Validation and Execution

Task 1: Check Account Balance Before Transfer - Write a PL/SQL block that takes an account number as input and displays the account balance.

Hint: Use `SELECT balance INTO` inside a PL/SQL block and `DBMS_OUTPUT.PUT_LINE` to display the balance.

```
SQL> DECLARE
  2     v_account_no  NUMBER := &account_number;
  3     v_balance     NUMBER;
  4 BEGIN
  5     SELECT balance INTO v_balance
  6     FROM accounts
  7     WHERE account_no = v_account_no;
  8
  9     DBMS_OUTPUT.PUT_LINE('Account Number: ' || v_account_no);
10     DBMS_OUTPUT.PUT_LINE('Current Balance: ' || v_balance);
11
12 EXCEPTION
13     WHEN NO_DATA_FOUND THEN
14         DBMS_OUTPUT.PUT_LINE('Error: Account not found.');
```

Enter value for account_number: 101

```
old 2:     v_account_no  NUMBER := &account_number;
new 2:     v_account_no  NUMBER := 101;
Account Number: 101
Current Balance: 4000

PL/SQL procedure successfully completed.
```

Task 2: Execute Fund Transfer Procedure - Call the `transfer_funds` procedure to transfer ₹500 from account 101 to account 102.

Hint: Use the `BEGIN...END;` block to execute the procedure.


```

6     v_from_balance NUMBER;
7 BEGIN
8     SELECT balance INTO v_from_balance
9     FROM accounts
10    WHERE account_no = p_from_account;
11
12    IF v_from_balance < p_amount THEN
13        RAISE_APPLICATION_ERROR(-20001, 'Insufficient fu
14    END IF;
15
16    UPDATE accounts
17    SET balance = balance - p_amount
18    WHERE account_no = p_from_account;
19
20    UPDATE accounts
21    SET balance = balance + p_amount
22    WHERE account_no = p_to_account;
23
24    COMMIT;
25
26    DBMS_OUTPUT.PUT_LINE('Transfer successful: ₹' || p_a
27 EXCEPTION
28    WHEN NO_DATA_FOUND THEN
29        DBMS_OUTPUT.PUT_LINE('Error: One or both account
30    WHEN OTHERS THEN
31        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
32 END;
33 /

```

Procedure created.

```

SQL> BEGIN
2     transfer_funds(101, 102, 500);
3 END;
4 /
Transfer successful: ₹500 from 101 to 102

```

Task 3: Validate Transaction Log - After executing the transfer, write an SQL query to display all transactions recorded in the `transactions` table.

Hint: Use `SELECT * FROM transactions;` to verify the transaction details.

```

SQL> SELECT * FROM transactions;

```

TRANSACTION_ID	FROM_ACCOUNT	TO_ACCOUNT	AMOUNT
1	101	102	1000
2	101	102	500

TRANSACTION_DATE

Task 4: Check Transaction History for a Specific Account

Write a PL/SQL block that takes an account number as input and displays all transactions (both sent and received) related to that account.

Hint: Use `SELECT * FROM transactions WHERE from_account = acc_no OR to_account = acc_no;` inside a PL/SQL block.

```
SQL> DECLARE
  2  v_acc_no NUMBER := &Enter_Account_Number;
  3 BEGIN
  4  FOR rec IN (SELECT * FROM transactions WHERE from_account = v_acc_no OR to_account = v_acc_no)
  5  LOOP
  6    DBMS_OUTPUT.PUT_LINE('Transaction ID: ' || rec.transaction_id ||
  7                          ', From: ' || rec.from_account ||
  8                          ', To: ' || rec.to_account ||
  9                          ', Amount: ₹' || rec.amount ||
  10                         ', Date: ' || rec.transaction_date);
  11  END LOOP;
  12 EXCEPTION
  13  WHEN NO_DATA_FOUND THEN
  14    DBMS_OUTPUT.PUT_LINE('No transactions found for this account.');
```

15 END;

16 /

Enter value for enter_account_number: 101

old 2: v_acc_no NUMBER := &Enter_Account_Number;

new 2: v_acc_no NUMBER := 101;

Transaction ID: 1, From: 101, To: 102, Amount: 71000, Date: 03-APR-25

02.28.47.845000 PM

PL/SQL procedure successfully completed.

Task 5: Prevent Self-Transfer

Modify the `transfer_funds` procedure to prevent an account from transferring money to itself. If the sender and receiver accounts are the same, raise an error message.

Hint: Add a condition inside the procedure:

`IF p_from_acc = p_to_acc THEN`

`RAISE_APPLICATION_ERROR(-20004, 'Sender and receiver cannot be the same.');`

`END IF;`

```

SQL> CREATE OR REPLACE PROCEDURE transfer_funds(
  2   p_from_acc NUMBER,
  3   p_to_acc NUMBER,
  4   p_amount NUMBER
  5 ) AS
  6   v_balance NUMBER;
  7 BEGIN
  8   IF p_from_acc = p_to_acc THEN
  9     RAISE_APPLICATION_ERROR(-20004, 'Sender and receiver accounts cannot be the same');
 10   END IF;
 11
 12   SELECT balance INTO v_balance FROM accounts WHERE account_no = p_from_acc;
 13
 14   IF v_balance < p_amount THEN
 15     RAISE_APPLICATION_ERROR(-20001, 'Insufficient balance');
 16   END IF;
 17
 18   UPDATE accounts SET balance = balance - p_amount WHERE account_no = p_from_acc;
 19   UPDATE accounts SET balance = balance + p_amount WHERE account_no = p_to_acc;
 20
 21   INSERT INTO transactions (from_account, to_account, amount, transaction_date)
 22   VALUES (p_from_acc, p_to_acc, p_amount, SYSDATE);
 23
 24   COMMIT;
 25   DBMS_OUTPUT.PUT_LINE('Transfer successful.');
```

Task 6: Create a Function to Check Account Balance

Write a PL/SQL function named `get_balance` that takes an account number as input and returns the current balance.

Hint:

```

CREATE OR REPLACE FUNCTION get_balance(p_acc_no NUMBER) RETURN NUMBER
AS
    v_balance NUMBER;
BEGIN
    SELECT balance INTO v_balance FROM accounts WHERE account_no =
p_acc_no;
    RETURN v_balance;
END;
/
```

Call it using:

```
SELECT get_balance(101) FROM dual;
```

```
SQL> CREATE OR REPLACE FUNCTION get_balance(p_acc_no NUMBER) RETURN NUMBER AS
 2     v_balance NUMBER;
 3 BEGIN
 4     SELECT balance INTO v_balance FROM accounts WHERE account_no = p_acc_no;
 5     RETURN v_balance;
 6 EXCEPTION
 7     WHEN NO_DATA_FOUND THEN
 8         RETURN NULL; -- Return NULL if account does not exist
 9 END;
10 /

Function created.
```

Task 7: Implement a Transfer Limit

Modify the `transfer_funds` procedure to set a maximum transfer limit of ₹10,000 per transaction. If a user tries to transfer more than this amount, raise an error.

Hint: Add a condition:

```
IF p_amount > 10000 THEN
    RAISE_APPLICATION_ERROR(-20005, 'Transfer amount exceeds the limit
of ₹10,000.');
```

```
END IF;
```

```

SQL> CREATE OR REPLACE PROCEDURE transfer_funds(
  2   p_from_acc NUMBER,
  3   p_to_acc NUMBER,
  4   p_amount NUMBER
  5 ) AS
  6   v_balance NUMBER;
  7 BEGIN
  8   -- Prevent self-transfer
  9   IF p_from_acc = p_to_acc THEN
10     RAISE_APPLICATION_ERROR(-20004, 'Sender and receiver cannot be the same.');
```

```

11   END IF;
12
13   -- Check transfer limit
14   IF p_amount > 10000 THEN
15     RAISE_APPLICATION_ERROR(-20005, 'Transfer amount exceeds the limit of ₹10,000.');
```

```

16   END IF;
17
18   -- Retrieve sender's balance
19   SELECT balance INTO v_balance FROM accounts WHERE account_no = p_from_acc;
20
21   -- Check if sender has sufficient balance
22   IF v_balance < p_amount THEN
23     RAISE_APPLICATION_ERROR(-20001, 'Insufficient balance.');
```

```

24   END IF;
25
26   -- Deduct amount from sender
27   UPDATE accounts SET balance = balance - p_amount WHERE account_no = p_from_acc;
28
29   -- Add amount to receiver
30   UPDATE accounts SET balance = balance + p_amount WHERE account_no = p_to_acc;
31
32   -- Log transaction
33   INSERT INTO transactions (from_account, to_account, amount, transaction_date)
34   VALUES (p_from_acc, p_to_acc, p_amount, SYSDATE);
35
36   -- Commit transaction
37   COMMIT;
38   DBMS_OUTPUT.PUT_LINE('Transfer successful.');
```

```

39
40 EXCEPTION
41   WHEN NO_DATA_FOUND THEN
42     RAISE_APPLICATION_ERROR(-20002, 'Invalid account number.');
```

```

43   WHEN OTHERS THEN
44     ROLLBACK;
45     RAISE_APPLICATION_ERROR(-20003, 'Transaction failed: ' || SQLERRM);
46 END;
47 /

```

Procedure created.

Task 8: Generate a Monthly Statement

Write a PL/SQL procedure that takes an account number and a month-year (e.g., 04-2025) as input and displays all transactions for that month.

Hint: Use `TO_CHAR(transaction_date, 'MM-YYYY')` in the `WHERE` clause:

```

SELECT * FROM transactions
WHERE (from_account = acc_no OR to_account = acc_no)
AND TO_CHAR(transaction_date, 'MM-YYYY') = '04-2025';

```

```

SQL> CREATE OR REPLACE PROCEDURE monthly_statement(
  2     p_acc_no NUMBER,
  3     p_month_year VARCHAR2
  4 ) AS
  5 BEGIN
  6     FOR rec IN (SELECT * FROM transactions
  7                 WHERE (from_account = p_acc_no OR to_account = p_acc_no)
  8                 AND TO_CHAR(transaction_date, 'MM-YYYY') = p_month_year)
  9     LOOP
 10         DBMS_OUTPUT.PUT_LINE('Transaction ID: ' || rec.transaction_id ||
 11                               ', From: ' || rec.from_account ||
 12                               ', To: ' || rec.to_account ||
 13                               ', Amount: ₹' || rec.amount ||
 14                               ', Date: ' || rec.transaction_date);
 15     END LOOP;
 16 EXCEPTION
 17     WHEN NO_DATA_FOUND THEN
 18         DBMS_OUTPUT.PUT_LINE('No transactions found for the given period.');
```

19 END;

20 /

```

SQL> BEGIN
  2     monthly_statement(101, '04-2025');
  3 END;
  4 /
```

PL/SQL procedure successfully completed.